Ali Rıza Aynacı

Istanbul/Turkey | aynacialiriza@gmail.com | +90 539 232 56 82
linkedin.com/in/alirizaaynaci | github.com/AliRizaAynaci | www.aynacialiriza.me

Education

Erciyes University, BS in Computer Engineering (3rd Year) *Expected Graduation: July 2027*

- **Head of Activities & Events Department**, *Erciyes University Software Technologies Club* (2025–Present)
- **Member & Embedded Software Lead**, *TEKNOFEST Rocket Team* (2023–2025)
  - Engineered 5Hz real-time flight software to process 16 telemetry signals across 5+ sensors, managing critical flight data and autonomous recovery logic.
- **English Proficiency:** B2 (Upper-Intermediate)

Technical Skills

Go · Java · Spring Boot · Kafka · WebSockets · REST API · PostgreSQL · NoSQL · SQL · Redis · Docker · GitHub Actions
Prometheus & Grafana · Hexagonal Architecture · Onion Architecture · CQRS · Rate Limiting · Git & GitHub · Linux

Projects

GoRL – High-Performance Rate Limiting Library | **Open Source** | ★ **138+ GitHub Stars** | github.com/AliRizaAynaci/gorl

Tech Stack: Go, Redis, Prometheus, GitHub Actions

- Designed and shipped a **production-ready** Go rate limiting library — Fixed Window, Sliding Window, Token Bucket, Leaky Bucket → **Benchmarked at 103–955 ns/op** across single and multi-key workloads (AMD Ryzen 7 4800H); **95%+ test coverage** and CI/CD via GitHub Actions
- Introduced **resource-scoped limiting** (v2.2.0): independent policies per endpoint (e.g. login: 5 req/min, search: 50 req/s) via shared backend, configurable from JSON/YAML — fully backward-compatible
- Architected minimal **Storage** interface with lock-free in-memory backend (`sync.Map` + atomic CAS) and Redis; built plug-and-play middleware for **net/http, Fiber, Gin, and Echo**

Liftio – AI-Powered Fitness Mobile App | **Full Stack** | *In Development* | github.com/AliRizaAynaci/liftio-backend

Tech Stack: Flutter, Go (Fiber v2), Kafka, WebSockets, Redis, SQLite, GoRL, Docker

- **Utilized Hexagonal Architecture (Ports & Adapters)** to decouple core logic, enabling a seamless **Fiber v2** migration; led a 6-stage optimization under simulated load tests of **1,000+** concurrent users, reducing system-wide latency by **52%–87%** and increasing throughput by **3–8x**.
- **Integrated custom rate-limiter (GoRL)** with Redis to early-reject **25k+ malicious Req/s**, preventing bcrypt-induced CPU starvation and improving collateral latency on read endpoints by **37%**.
- **Replaced legacy polling** with an event-driven **Kafka & WebSockets** architecture, decoupling async workflows and eliminating redundant load from repeated client requests.
- **Resolved SQLite lock contention** by enabling **WAL mode** and connection pooling, reducing latency by **52%–87%** and increasing throughput by **2.1x–8.3x** across core endpoints.
- **Improved read-heavy query performance** via composite indexing, reaching **26,371 Req/s** peak throughput at **37ms** latency.

Achievements

NASA Space Apps Challenge 2025 - Global Nominee (Team Lead)

- **Led a team of 6** to rank in the **top 11% globally** among 114,000+ participants and 11,500+ projects.
- **Engineered a concurrent Go backend** in 48h to process NASA Earth Observation Data via Fiber and PostgreSQL.
- **Deployed and coordinated** Dockerized backend, frontend, and Python-based ML services, enabling end-to-end data flow and real-time visualization via a React dashboard.
- *GitHub Links: Backend | Frontend | ML Pipeline*

Technical Writings

- **Hexagonal Architecture: Concepts, Motivation, and Implementation in Go**
- **Redis Sharding in Go: From Single Node Bottleneck to Horizontal Scale**
- **Google OAuth2 Login in Go: A Minimal and Complete Guide**
- **Designing a Scalable Rate Limiter**
- **Concurrency and Synchronization in Go**